

Module 03

Q1 : Explain how Dynamic Programming works with an example. Mention its steps and applications.

Dynamic Programming (DP) is a technique used to solve problems by breaking them into overlapping subproblems and solving each only once, storing the results for future use. It is used when a problem has **optimal substructure** (solution can be built from solutions of subproblems) and **overlapping subproblems** (same subproblems are solved multiple times).

Steps in Dynamic Programming:

1. **Divide the complex problem** into smaller subproblems.
2. **Find optimal solutions** to each subproblem.
3. **Store the results** of these subproblems (called **memoization**).
4. **Reuse stored results** instead of recomputing.
5. **Build the final solution** from the subproblem results.

```
def fib(n):
    dp = [0, 1]
    for i in range(2, n+1):
        dp.append(dp[i-1] + dp[i-2])
    return dp[n]
```

Q2: Explain Floyd's Warshall Algorithm with its advantages and time complexity.

Floyd Warshall Algorithm is a well-known algorithm used to solve the **All-Pairs Shortest Path Problem** in a graph. It finds the shortest path between every pair of vertices.

It is an example of the **Dynamic Programming approach**, as it builds solutions step by step by using already known smaller solutions.

☑ **Working:**

- It checks for every possible path between all pairs of nodes.
- If a shorter path is found through another node, it updates the distance.
- It uses a **distance matrix** and updates it after comparing paths through each node.

☑ **Advantages:**

- Very **simple and easy to implement**.
- Works well even for **dense graphs**.
- Can handle **negative edge weights** (but not negative cycles).

☑ **Time Complexity:**

- The algorithm has **three nested loops** over all nodes.
- Each loop runs **n times**, so total time is **O(n³)**.
- n is the number of vertices in the graph.

Calculate-M1

$$A^k[i,j] = \min \{ A^{k-1}[i,j], A^{k-1}[i,k] + A^{k-1}[k,j] \}$$

• **M1**

	1	2	3	4
1	0	3	∞	7
2	8	0		
3	5		0	
4	2			0

M0

	1	2	3	4
1	0	3	∞	7
2	8	0	2	∞
3	5	∞	0	1
4	2	∞	∞	0

$$M1[2,3] = \min \{ M0[2,3], M0[2,1] + M0[1,3] \} = \min \{ 2, 8 + \infty \} = 2$$

$$M1[2,4] = \min \{ M0[2,4], M0[2,1] + M0[1,4] \} = \min \{ \infty, 8 + 7 \} = 15$$

$$M1[3,2] = \min \{ M0[3,2], M0[3,1] + M0[1,2] \} = \min \{ \infty, 5 + 3 \} = 8$$

$$M1[3,4] = \min \{ M0[3,4], M0[3,1] + M0[1,4] \} = \min \{ 1, 5 + 7 \} = 1$$

$$M1[4,2] = \min \{ M0[4,2], M0[4,1] + M0[1,2] \} = \min \{ \infty, 2 + 3 \} = 5$$

$$M1[4,3] = \min \{ M0[4,3], M0[4,1] + M0[1,3] \} = \min \{ \infty, 2 + \infty \} = \infty$$

ANS: M1

↓

	1	2	3	4
1	0	3	∞	7
2	8	0	2	15
3	5	8	0	1
4	2	5	∞	0

Q3: Backtracking

Backtracking algorithms are problem-solving strategies that explore multiple options in order to find the best solution. They work by incrementally testing different paths; when one path leads to a dead end, the algorithm backtracks and tries an alternative until the correct solution is found. It's akin to solving a puzzle by testing different pieces until they fit perfectly.

Backtracking is a technique that involves finding solutions by trying various possibilities and undoing choices when they lead to a dead end. This process continues until a viable solution is discovered or all options have been explored. It is especially useful in problems where multiple potential solutions must be tested, such as:

- Searching for a path in a maze
- Solving puzzles like Sudoku

When a dead end is reached, the algorithm backtracks to the previous decision point and explores a different path. This process continues until a solution is found or all paths have been exhausted.

Applications of Backtracking:

1. N-Queen Problem
2. Sum of Subsets Problem
3. Graph Coloring
4. Hamiltonian Cycle

Q4: How Does a Backtracking Algorithm Work?

A backtracking algorithm operates by recursively exploring all possible solutions to a problem. Here's a breakdown of its general working process:

1. **Choose an initial solution.**
2. **Explore all possible extensions of the current solution.**
3. **If an extension leads to a solution, return that solution.**
4. **If an extension does not lead to a solution, backtrack to the previous solution and attempt a different extension.**
5. **Repeat steps 2–4 until all possible solutions have been explored.**

This algorithm works step by step, and the process of "backtracking" ensures that all possible paths are evaluated, ensuring the most optimal solution is found.

Key Terms Related to Backtracking:

- **Live Node:** A node that can still generate further child nodes and remains part of the search process.
- **E Node:** A node where children are being generated, and it's a point where further solutions may emerge.
- **Success Node:** A node that provides a feasible solution and satisfies the problem's constraints.
- **Dead Node:** A node that cannot be expanded further and does not lead to a feasible solution.

Q5 N-Queens Problem

The N-Queens problem is a classical problem in which we are tasked with placing n queens on an $n \times n$ chessboard in such a way that no two queens can attack each other. The queens should be placed such that:

- No two queens share the same row.
- No two queens share the same column.
- No two queens share the same diagonal.

Solution Insights:

- For $n = 1$, the problem has a trivial solution (a single queen on a 1×1 board).
- For $n = 2$ and $n = 3$, no solutions exist.
- The problem becomes non-trivial and interesting for $n \geq 4$.

4-Queens Problem (Example):

Consider a 4×4 chessboard. We need to place 4 queens such that they do not attack each other. Since each queen must occupy a different row, we can think of the problem as selecting one column for each queen. The working algorithm for this problem involves recursively attempting to place each queen in a column of each row, ensuring that no two queens threaten each other (i.e., no two queens are placed in the same column or diagonal).

Backtracking Approach for the 4-Queens Problem:

1. **Start in the first row** and try placing the queen in each column one by one.
2. **Move to the next row**, and for each column, check whether it is safe to place the queen (i.e., ensure that no other queen is in the same column or diagonal).
3. **If a safe position is found**, place the queen and move to the next row.
4. **If no safe position is available**, backtrack to the previous row and try the next possible position for the queen there.
5. **Repeat** until all queens are placed successfully, or all possibilities are explored.

Example Solution for a 4×4 Board:

```
Q . . .  
. . . Q  
. Q . .  
. . Q .
```

In this arrangement, all 4 queens are placed in such a way that they do not attack each other.

Q6 Subset-Sum Problem

The Subset-Sum problem asks if there is a subset of a given set $A = \{a_1, a_2, \dots, a_n\}$ of n positive integers whose sum is equal to a given positive integer d . The goal is to find a subset of A such that the sum of the elements in the subset is exactly d .

For example, given a set $A = \{3, 34, 4, 12, 5, 2\}$ and $d = 9$, we need to find a subset whose sum is 9.

Algorithm (Dynamic Programming Approach):

1. **Create a 2D table** $dp[i][j]$ where $dp[i][j]$ is True if there is a subset of the first i elements of A that sums to j . Otherwise, $dp[i][j]$ is False.
2. **Base Case:** $dp[0][0]$ is True, meaning a sum of 0 can always be achieved by choosing no elements.
3. **Recursive Case:** For each element $A[i]$, and for each sum j , check if:
 - The sum j can be achieved either by excluding the element $A[i]$ or by including $A[i]$ (i.e., $dp[i-1][j]$ or $dp[i-1][j - A[i]]$).
4. **Final Step:** If $dp[n][d]$ is True, then there is a subset that sums to d .

Example:

Given $A = \{3, 34, 4, 12, 5, 2\}$ and $d = 9$, we want to find a subset that sums to 9.

Step-by-step Execution:

1. Start with an empty subset and begin to fill the DP table.
2. Build the table row by row, checking for each element if it can contribute to the sum.
3. The final DP table will indicate whether a subset that sums to 9 exists.

After running the algorithm, we can determine that the subset $\{4, 5\}$ sums to 9.

Q8 Traveling Salesman Problem (TSP):

The **Traveling Salesman Problem (TSP)** is a classical optimization problem in the field of computer science and mathematics. It can be described as follows:

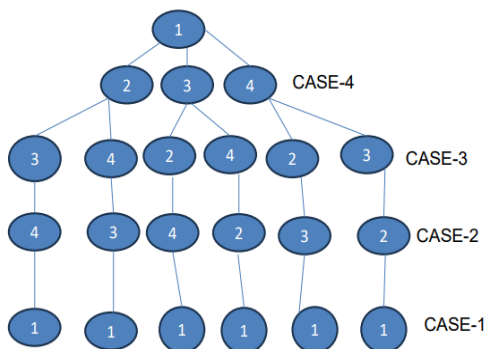
- **Given:**
 - A set of **cities** (or nodes).
 - The **distance** between every pair of cities (or edges).
- **Objective:** A **salesman** must visit **each city exactly once** and then return to the starting city. The task is to determine the **shortest possible route** that allows the salesman to visit all the cities and return to the start.

In simpler terms, TSP is a route optimization problem where the goal is to find the least costly way to visit each city and return to the origin, ensuring the route is minimal in terms of travel distance or time.

Key Points:

- The salesman must **visit every city exactly once**.
- The salesman **returns to the starting city** at the end of the journey.
- The objective is to minimize the total distance (or cost) traveled.

TSP is widely used in logistics, route planning, and operations research and is known for being **NP-hard**, meaning finding an optimal solution is computationally challenging as the number of cities grows.



🔍 All Possible Tours from Node 1:

1. $1 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow 1$
 $= 10 + 9 + 12 + 8 = 39$
2. $1 \rightarrow 2 \rightarrow 4 \rightarrow 3 \rightarrow 1$
 $= 10 + 10 + 9 + 6 = 35$
3. $1 \rightarrow 3 \rightarrow 2 \rightarrow 4 \rightarrow 1$
 $= 15 + 13 + 10 + 8 = 46$
4. $1 \rightarrow 3 \rightarrow 4 \rightarrow 2 \rightarrow 1$
 $= 15 + 12 + 8 + 5 = 40$
5. $1 \rightarrow 4 \rightarrow 2 \rightarrow 3 \rightarrow 1$
 $= 20 + 8 + 9 + 6 = 43$
6. $1 \rightarrow 4 \rightarrow 3 \rightarrow 2 \rightarrow 1$
 $= 20 + 9 + 13 + 5 = 47$

EXAMPLE

TSP

Solve the following TSP which is represented as a directed graph and whose edge lengths are given by cost distance matrix

	1	2	3	4
1	0	10	15	20
2	5	0	9	10
3	6	13	0	12
4	8	8	9	0

Optimal Route:

1 → 2 → 4 → 3 → 1 with total cost = 35

Q7 What is Greedy Method? (5 Marks)

The **Greedy Method** is a problem-solving technique where the algorithm makes a **locally optimal** choice at each step with the hope that these choices lead to a **globally optimal** solution. It **does not** reconsider its decisions — once it makes a move, it sticks with it.

Think of it like grabbing the best treasure you see first, hoping it leads to the biggest loot — but sometimes, the real jackpot was deeper in the cave.

Key Features:

- **Builds solution step-by-step.**
- **Doesn't backtrack.**
- **Chooses the best option available at each step.**

Advantages of Greedy Approach:

1. **Simplicity** – Easy to write, read, and debug.
2. **Speed** – Often faster than other complex methods like Dynamic Programming.
3. **Efficient for specific problems** – Like **Prim's**, **Kruskal's**, **Dijkstra's** algorithms.
4. **Less Memory Use** – Doesn't need to store all solutions.
5. **Real-Time Systems** – Perfect for quick decisions where precision isn't everything.

Limitations:

It **doesn't** always **give the optimal solution** — especially when local choices don't align with global goals.

Q8: Types of Graphs, Spanning Tree, Properties, and MST Methods (10M)

→ Graph:

A **graph** is a collection of **vertices (nodes)** connected by **edges**. It represents systems like road maps, communication lines, and networks.

→ Types of Graphs:

- **Undirected Graph:**
Edges have **no direction**; connections are **bidirectional**.
Example: Friendship in social networks.
- **Directed Graph (Digraph):**
Edges have a **specific direction**.
Example: Following someone on Instagram.
- **Connected Graph:**
A path exists between **every pair of vertices**.
No node is isolated; the graph forms a single connected structure.

→ Spanning Tree:

A **spanning tree** is a **subgraph** of a **connected, undirected graph** that:

- Includes **all vertices**
- Contains **no cycles**
- Uses exactly **(n - 1)** edges for *n* vertices
- Maintains **connectivity** with minimal edges

- May or may not have **weights** on edges

💡 For a complete graph with n nodes, total possible spanning trees = n^{n-2}

Example: $n = 5 \rightarrow 5^3 = 125$ spanning trees

→ Properties of Spanning Tree:

- No **cycles** allowed
- Always has **(n - 1)** edges
- **Removing** any edge disconnects the graph
- **Adding** any edge creates a cycle
- Exists only for **connected graphs**
- A graph can have **multiple spanning trees**
- **Maximum removable edges** = $e - n + 1$
(e = original edges, n = vertices)

→ Minimum Spanning Tree (MST):

An **MST** is a spanning tree with the **lowest total edge weight**.

It connects all nodes with **minimum cost, distance, or load** — perfect for real-world networks like roads, pipelines, or telecom cables.

→ Methods to Find MST:

- **Prim's Algorithm:**
Start from any vertex, pick the **minimum edge** that expands the tree, repeat until all nodes are included.
- **Kruskal's Algorithm:**
Sort all edges by **weight**, keep adding the smallest one **without forming cycles**, until the tree has $(n - 1)$ edges.

Q9: Explain Prim's Algorithm with steps, theory, and logic (10M)

→ Definition:

Prim's Algorithm is a **greedy algorithm** used to find a **Minimum Spanning Tree (MST)** in a **connected, undirected, weighted graph**.

It builds the MST **incrementally**, starting with a single vertex and **expanding** by choosing the **smallest possible edge** that connects a new vertex to the growing tree.

The goal:

To include **all vertices** in the tree with **minimum total edge weight**, and **no cycles**.

Prim's algorithm maintains **two sets of vertices**:

1. **MST Set:** Vertices **already included** in the MST.
2. **Non-MST Set:** Vertices **not yet included**.

At each step, it finds the **lightest edge** connecting these two sets and **adds the new vertex** to the MST set.

→ Working Mechanism:

1. Start with any **random vertex** (commonly the first node).
2. Assign **key values** to each vertex:
 - Start vertex → **key = 0**

- All others → **key** = ∞
- 3. Pick the **vertex with the smallest key** that's not yet in MST.
- 4. Include it in the MST set.
- 5. For all its adjacent vertices:
 - If the **edge weight is smaller** than their current key, **update the key** and set the current vertex as the parent.
- 6. Repeat the process until **all vertices are included** in the MST.
- 7. Cycle or loops should not be formed

→ Example (Conceptual):

Suppose a graph with vertices A, B, C, D.

- Start from A → include A in MST
- Among edges from A, pick the **minimum edge**, say A–C
- Include C → look at all edges from A and C
- Pick the smallest again, say C–B
- Continue this way until all nodes are covered

→ Properties:

- Builds **one tree**, node by node.
- Always picks the **greedy choice** (minimum weight).
- Does **not form cycles**.
- Guaranteed to return an MST if the graph is connected.

→ Applications:

- Network design (LAN, WAN, telecom)
- Road and pipeline construction
- Clustering algorithms

→ Pseudocode (Lightweight):

Initialize all key values to ∞

Set `key[start_vertex] = 0`

While MST does not include all vertices:

Pick vertex `u` with minimum key not in MST

Add `u` to MST

For each neighbor `v` of `u`:

If `v` not in MST and `weight(u,v) < key[v]`:

Update `key[v] = weight(u,v)`

Set parent of `v` to `u`

Q10: Explain Kruskal's Algorithm with steps and logic. (10M)

→ Definition:

Kruskal's Algorithm is a **greedy algorithm** used to find the **Minimum Spanning Tree (MST)** of a **connected, undirected, weighted graph**.

It works by always choosing the **minimum weight edge** that **doesn't form a cycle**, and continues until all vertices are connected.

→ Working Mechanism of Kruskal's Algorithm:

The algorithm follows a **greedy approach**, where it selects the **smallest possible edge** at each step to build the MST, ensuring that the tree remains **cycle-free** and eventually includes all vertices.

Steps:

1. **Sort all edges:**
 - The algorithm begins by **sorting all the edges** of the graph in **increasing order of their weights**. This helps in always picking the smallest edge first.
2. **Create a forest:**
 - Initially, treat every vertex as a **separate tree** (i.e., a disjoint set of individual vertices). This is done using a **disjoint set** data structure (Union-Find) to track which vertices are in which tree.
3. **Iterate over sorted edges:**
 - Process edges in the sorted order, starting with the smallest edge.
4. **Add edges to the MST:**
 - For each edge **(u, v)**, check if **u** and **v** are in the same tree or not:
 - If they belong to **different trees**, **add the edge to the MST** and **merge the two trees** into a single tree. This means you are connecting two previously separate parts of the graph.
 - If they belong to the **same tree**, **skip the edge** because adding it would form a cycle.
5. **Stop when MST is complete:**
 - The algorithm continues this process until the MST contains **(V - 1) edges**, where **V** is the total number of vertices in the graph. At this point, the MST is complete.
6. **End the algorithm:**
 - Once the MST is constructed, or when there are no more edges to process, the algorithm terminates.

→ Cycle Detection:

- The **key aspect** of Kruskal's Algorithm is to avoid adding edges that would create a cycle.
- To detect cycles, the **Union-Find data structure** is used, which helps in efficiently determining whether two vertices are in the same tree (or set).
 - **Union operation:** Connects two sets (trees) together.
 - **Find operation:** Determines the root of a set (tree) to check if two vertices belong to the same set.

Q11 Theory: Single Source Shortest Paths

Shortest Path Problem deals with finding the shortest path between two vertices in a **weighted, directed graph**. A graph is represented as $G=(V,E)$ $G = (V, E)$, where:

- V is the set of vertices (nodes).
- E is the set of edges (connections between nodes).
- Each edge has a **weight** (or cost), denoted as $w:E \rightarrow \mathbb{R}$: $E \rightarrow \mathbb{R}$, that maps each edge to a real-valued weight.

The goal is to find the **shortest path** from a source vertex $s \in V$ to a target vertex $t \in V$. The weight of a **path** $p=(v_0,v_1,...,v_k)$ $p = (v_0, v_1, ..., v_k)$ is the sum of the weights of the edges that make up the path, i.e., $w(p)=w(v_0,v_1)+w(v_1,v_2)+...+w(v_{k-1},v_k)$ $w(p) = w(v_0, v_1) + w(v_1, v_2) + ... + w(v_{k-1}, v_k)$.

The **shortest path weight** from u to v , denoted as $\delta(u,v)$, is the **minimum weight** of any path from u to v :

$$\delta(u,v)=\min(w(p):u \rightarrow v)$$

If no path exists, $\delta(u,v)=\infty$

The **single source shortest path problem** asks to find the shortest path from a given **source vertex ss** to **all other vertices** in the graph. This means calculating the shortest path to every vertex $v \in V$.

Variants of the Shortest Path Problem:

1. **Single-Destination Shortest Path Problem:**
 - The problem here is to find the shortest path to a **given destination vertex tt** from all other vertices in the graph.
 - **Solution:** By reversing the direction of all edges in the graph, this problem can be transformed into a **single-source shortest path problem** from the destination tt.
2. **Single-Pair Shortest Path Problem:**
 - This variant involves finding the shortest path from a **specific source vertex uu** to a **specific target vertex vv**.
 - **Solution:** This can be addressed by solving a **single-source shortest path problem** from uu to determine the shortest path to vv.
3. **All-Pairs Shortest Path Problem:**
 - In this case, the goal is to find the shortest path between **every pair of vertices** in the graph.
 - **Solution:** A straightforward approach would be to run a **single-source shortest path algorithm** from **each vertex** in the graph. However, more efficient algorithms exist to solve this problem, such as **Floyd-Warshall** or **Johnson's algorithm**, which generally perform better in terms of time complexity.

Applications of Shortest Path Problems:

The **Single Source Shortest Path** problem has a wide range of applications in real-world scenarios such as:

- **Network routing** (e.g., finding the shortest path for data transmission in computer networks).
- **Navigation systems** (e.g., GPS finding the shortest driving route).
- **Transport and logistics** (e.g., shortest paths for trucks or vehicles on a delivery route).

Key Algorithms for Solving Single Source Shortest Path Problem:

1. **Dijkstra's Algorithm:**
 - **Use Case:** Best suited for graphs with **non-negative weights**.
 - **Time Complexity:** $O(V \log V + E)$ using a priority queue (min-heap).
 - **Approach:** It iteratively picks the **vertex with the smallest tentative distance** and updates the shortest distances to its adjacent vertices.
2. **Bellman-Ford Algorithm:**
 - **Use Case:** Can handle graphs with **negative weight edges**, but not negative weight cycles.
 - **Time Complexity:** $O(V \cdot E)$
 - **Approach:** It **relaxes edges** repeatedly, ensuring that the shortest path is found after $V-1$ iterations (where V is the number of vertices).
3. **Floyd-Warshall Algorithm:**
 - **Use Case:** Typically used for the **All-Pairs Shortest Path** problem.
 - **Time Complexity:** $O(V^3)$
 - **Approach:** A dynamic programming method that computes shortest paths between all pairs of vertices in a single run.

4. A Algorithm*:

- **Use Case:** Efficient for finding the shortest path in **graphs with heuristics** (often used in pathfinding on maps).
- **Time Complexity:** Depends on the heuristic and implementation.

Q12 Dijkstra's Algorithm: Theory and Working Mechanism (IMP)

Dijkstra's Algorithm is a well-known algorithm for solving the **single-source shortest path problem**. This means it finds the shortest path from a starting node (source) to every other node in a **weighted graph** with **non-negative weights**.

The algorithm operates on the **greedy approach**, where it iteratively picks the node with the smallest known distance from the source, processes it, and updates the distances of its neighboring nodes.

Key Characteristics:

- **Greedy Algorithm:** It always picks the next node that is closest to the source, trying to find the locally optimal solution at each step.
- **Graph Requirements:** Works only on graphs with **positive edge weights**.
- **Time Complexity:**
 - With an **adjacency matrix**, it runs in $O(V^2)$, where V is the number of vertices.
 - Using an **adjacency list** with a **min-heap** (priority queue), it can be optimized to $O((V+E) \log V)$, where E is the number of edges.

The algorithm uses two key properties for each vertex:

1. **Visited Property:** Marks whether the node has been processed (i.e., its shortest path is finalized).
2. **Path Property:** Stores the shortest known path from the source node to this vertex.

Working Process:

1. **Initialization:**
 - Mark all vertices as unvisited.
 - Set the distance to the source node to 0 and all other nodes to infinity (∞).
2. **Start at the Source:**
 - Begin with the source vertex and update the distance to its neighbors.
3. **Relaxation:**
 - For each neighboring node of the current vertex, update its shortest distance if a shorter path is found (i.e., through the current node).
 - This is known as **relaxation**:
$$p[n] = \min(\text{cost}, \text{weight} + \text{existing cost})$$
$$p[n] = \min(\text{cost}, \text{weight} + \text{existing cost})$$
 - Here, $p[n]$ is the current path to node n , and the relaxation checks if going through the current node provides a shorter path.
4. **Selecting the Next Node:**
 - After updating the neighbors, mark the current node as visited.
 - Select the unvisited node with the smallest tentative distance and repeat the process until all nodes are visited or all reachable nodes have their shortest paths finalized.
5. **Termination:**
 - The process stops when all nodes are marked as visited, meaning all shortest paths have been computed.

Dijkstra's Algorithm with Example:

Let's break down the algorithm with a simple example:

Consider a graph with 5 vertices A,B,C,D,E and the following edges with weights:

- A→B weight 4
- A→C weight 5
- B→D weight 3
- C→D weight 2
- D→E weight 6

Step 1: Initialize the distances:

- Distance to source A=0
- All other nodes are set to infinity (∞):
A:0, B: ∞ , C: ∞ , D: ∞ , E: ∞

Step 2: Start with the source node A, visit its neighbors:

- A→B: Relax the edge, update B's distance to 4.
- A→C: Relax the edge, update C's distance to 5.
- **Updated Distances:**
A:0, B:4, C:5, D: ∞ , E: ∞

Step 3: Mark A as visited, choose the unvisited node with the smallest distance: **B** (distance = 4).

- B→D: Relax the edge, update D's distance to 7 (4 + 3).
- **Updated Distances:**
A:0, B:4, C:5, D:7, E: ∞

Step 4: Mark B as visited, choose the next unvisited node with the smallest distance: **C** (distance = 5).

- C→D: Relax the edge, update D's distance to 7 (no change since the current path to D is already 7).
- **Updated Distances:**
A:0, B:4, C:5, D:7, E: ∞

Step 5: Mark C as visited, choose the next unvisited node with the smallest distance: **D** (distance = 7).

- D→E: Relax the edge, update E's distance to 13 (7 + 6).
- **Updated Distances:**
A:0, B:4, C:5, D:7, E:13

Step 6: Mark D as visited, the last unvisited node is **E**, but no updates are made as all its neighbors are already visited.

Final Shortest Paths:

- A→A =0
- A→B =4
- A→C =5
- A→D =7
- A→E =13

Q13 Bellman-Ford algorithm Algorithm: Theory and Working Mechanism (IMP)

The Bellman-Ford algorithm is another approach to finding the shortest path in a graph, particularly useful when the graph contains edges with negative weights. Unlike Dijkstra's algorithm, which doesn't handle negative weights well, Bellman-Ford guarantees the correct shortest path calculation even in such cases. Moreover, it can detect negative weight cycles, which Dijkstra's algorithm cannot.

Key Points of Bellman-Ford:

- **Single-Source Shortest Path Algorithm:** It finds the shortest path from a source vertex to all other vertices.
- **Works with Negative Weights:** Bellman-Ford handles graphs with negative edge weights, unlike Dijkstra's.
- **Relaxation Process:** The algorithm relaxes all the edges $n - 1$ times, where n is the number of vertices. This means it updates the shortest distance of each vertex by checking all edges in the graph.
- **Negative Cycle Detection:** If the algorithm can still relax any edge after $n - 1$ iterations, it means there's a negative weight cycle in the graph.

Algorithm Steps:

1. **Initialization:**
 - Set the distance to the source vertex as 0 and all other vertices as infinity (∞).
 - Keep track of the previous vertex for each vertex (to reconstruct the shortest path later).
2. **Relaxation:**
 - For each edge (u, v) in the graph, if the distance to v through u is shorter than the current known distance to v , update it.
 - Repeat this for $n - 1$ iterations (where n is the number of vertices).
3. **Negative Cycle Detection:**
 - After $n - 1$ iterations, if any edge can still be relaxed, it indicates a negative weight cycle, and the algorithm reports this.

Relaxation Formula:

For an edge (u, v) with weight w :

- If $\text{distance}[u] + w < \text{distance}[v]$, update $\text{distance}[v]$.

Example Walkthrough:

1. **First Iteration:**
 - Relax all edges once. If the relaxation reduces the distance to any vertex, update it.
2. **Subsequent Iterations:**
 - Repeat the relaxation process. By the $n-1$ th iteration, the shortest paths will have been found.

Vertices: A, B, C, D, E, F

Edges: (A, B, 6), (A, C, 4), (A, D, 5), (B, E, -1), (C, E, 3),
(D, C, -2), (D, F, -1), (E, F, 3), (C, B, -2)

For the source vertex A:

1. Initialize: Distance: A=0, B= ∞ , C= ∞ , D= ∞ , E= ∞ , F= ∞
2. **Relax all edges** in the first iteration:
 - (A, B) $\rightarrow$ Update B to 6.
 - (A, C) $\rightarrow$ Update C to 4.
 - (A, D) $\rightarrow$ Update D to 5.
 - (B, E) $\rightarrow$ Update E to 5.
 - (C, E) $\rightarrow$ No update for E.
 - (D, C) $\rightarrow$ Update C to 3 (since $5 - 2 < 4$).
 - (D, F) $\rightarrow$ Update F to 4.
 - (E, F) $\rightarrow$ No update for F.
 - (C, B) $\rightarrow$ Update B to 1 (since $3 - 2 < 6$).

After the first iteration: Distance: A=0, B=1, C=3, D=5, E=5, F=4

3. **Second iteration:** Relax all edges again. The distances stay the same, meaning no further updates are needed.
4. **Final Output:** The shortest distances from vertex A to all other vertices are:
A=0, B=1, C=3, D=5, E=5, F=4

Time Complexity:

- **Time Complexity:** $O(V * E)$ where:
 - V is the number of vertices.
 - E is the number of edges.
- This is because we relax all edges $V - 1$ times (one less than the number of vertices).

Negative Cycle Detection:

If, after the $V - 1$ iterations, there is still an edge that can be relaxed, a negative cycle is detected, and the algorithm should report an error.

Pseudocode:

```
function bellmanFord(G, S):
    # Initialize distances
    for each vertex V in G:
        distance[V] = infinity
        previous[V] = NULL
    distance[S] = 0

    # Relax edges V-1 times
    for i from 1 to V-1:
        for each edge (U, V) in G:
            if distance[U] + weight(U, V) < distance[V]:
                distance[V] = distance[U] + weight(U, V)
                previous[V] = U

    # Check for negative-weight cycles
    for each edge (U, V) in G:
        if distance[U] + weight(U, V) < distance[V]:
            return "Error: Negative Cycle Exists"

    return distance[], previous[]
```